CROYEZ CLOTHING

CONSULTANCY PROJECT REPORT

Submitted by

JAYA NIRENJAN A C

22CSR079

KAVINNATH A R

22CSR094

KESHORE V M

22CSR099

in partial fulfilment of the requirements for

the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

APRIL 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KONGU ENGINEERING COLLEGE

(Autonomous)

PERUNDURAI ERODE – 638 060

APRIL 2025

BONAFIDE CERTIFICATE

This is to certify that the Project report entitled **CROYEZ CLOTHING** is the bonafide record of the project work done by **JAYA NIRENJAN A C (Register No: 22CSR079), KANINNATH A R (Register No: 22CSR094) , KESHORE V M (Register No : 22CSR099),** in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering in **Computer Science and Engineering** of Anna University Chennai during the year 2024-2025.

SUPERVISOR **HEAD OF THE DEPARTMENT**

Date: **(Signature with seal)**

KONGU ENGINEERING COLLEGE

(**Autonomous**)

PERUNDURAI ERODE – 638 060

NOVEMBER 2024

DECLARATION

We affirm that the Project Report titled **CROYEZ CLOTHING** being submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering is the original work carried out by us. It has not formed the part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion this or any other candidate.

Date:

JAYA NIRENJAN A C

(22CSR079)

KAVINNATH A R

(**22CSR094)**

KESHORE V M

(22CSR099)

I certify that the declaration made by the above candidate is true to the best of my knowledge.

Date:

Name and Signature of the Supervisor

ABSTRACT

This project presents a comprehensive e-commerce solution for men's clothing through a web application named "Croyez Clothing." Designed with a user-centric approach, the application enables customers to browse, search, and purchase men's apparel with an intuitive interface, while providing administrators with robust tools to manage products, inventory, and orders. The front-end of the application is developed using React.js, along with HTML and CSS for a responsive and intuitive user interface. The back-end, built with Node.js and Express, handles the application's server-side logic, ensuring seamless data processing and secure communication between the client and the server. MongoDB serves as the primary database, providing a scalable, NoSQL solution for storing product data, user information, and transaction records. Key features of the Croyez Clothing platform include user authentication, product categorization, advanced filtering options, shopping cart functionality, secure payment processing via PayPal integration, and product rating systems. The administrative module offers comprehensive CRUD operations for product management, order tracking, and inventory control. This system offers flexibility and scalability, making it suitable for expanding product lines and accommodating growing customer bases. Overall, this project leverages modern web development technologies to deliver a functional, reliable, and aesthetically pleasing e-commerce platform that enhances the online shopping experience for men's fashion.

ACKNOWLEDGEMENT

We express our sincere thanks and gratitude to **Thiru. A. K. ILANGO B.Com., M.B.A., LLB.,** our beloved Correspondent, and all other philanthropic trust members of Kongu Vellalar Institute of Technology Trust who have always encouraged us in academic and co-curricular activities.

We are extremely thankful with no words of a formal nature to the dynamic Principal **Dr. V. BALUSAMY, M.Tech., Ph.D.,** for providing the necessary facilities to complete our work.

We would like to express our sincere gratitude to our respected Head of the Department **Dr. S. MALLIGA, M.E., Ph.D.,** for providing the necessary facilities.

We extend our thanks to **Ms. S. MOHANA PRIYA, M.E.,** Assistant Professor, Computer Science Engineering, Project Coordinator for her encouragement and valuable advice that made us carry out the project work successfully.

We extend our gratitude to our Supervisor **Ms. S. MOHANA PRIYA, M.E.,** Assistant Professor of Computer Science Engineering, for her valuable ideas and suggestions, which have been very helpful in the project. We are grateful to all the faculty members of the Computer Science and Engineering Department, for their support.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE No.
	ABSTRACT	iv
	LIST OF FIGURES	viii
1	**INTRODUCTION**	**1**
	1.1 EXISTING SYSTEM	1
	1.2 SYSTEM STUDY	2
	1.3 OBJECTIVE	3
	1.4 SCOPE	3
2.	**GENERAL DESCRIPTION**	**4**
	2.1 PROJECT PERSPECTIVE	4
	2.2 USER CHARACTERISTICS	4
	2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS	5
3.	**REQUIREMENTS**	**6**
	3.1 FUNCTIONAL REQUIREMENTS	6
	3.2 NON-FUNCTIONAL REQIREMENTS	7
4.	**DETAILED DESIGN**	**9**
	4.1 ARCHITECTURAL DESIGN	9
	4.1.1 MODULE CHARACTERISTICS	9
	4.1.1.1 USER MODULE	9

CHAPTER NO	TITLE	PAGE No.
	4.1.1.2 ADMIN MODULE	10
	4.1.1.3 USE CASE DIAGRAM	11
	4.1.1.4 SEQUENCE DIAGRAM	12
	4.2 INTERFACE DESIGN	13
	4.3 DATABASE DESIGN	13
	4.4 OUTPUT DESIGN	14
5.	**TESTING**	**15**
	5.1 UNIT TESTING	15
	5.2 REGRESSION TESTING	15
	5.3 VALIDATION TESTING	15
	5.4 VERIFICATION TESTING	16
	5.5 INTEGRATION TESTING	16
6.	**CONCLUSION AND FUTURE SCOPE**	**17**
	APPENDIX 1	18
	APPENDIX 2	36
	REFERENCES	43

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	FLOW DIAGRAM	10
4.2	USE CASE DIAGRAM	11
4.3	SEQUENCE DIAGRAM	12
4.4	DATABASE DESIGN DIAGRAM	14

CHAPTER 1

INTRODUCTION

In today's digital era, e-commerce has revolutionized the way consumers shop for clothing and accessories. An online men's clothing store serves as a valuable platform for customers to browse, select, and purchase apparel from the comfort of their homes. This project aims to develop a comprehensive, user-friendly e-commerce platform for men's clothing named "Croyez Clothing" using a full-stack approach. The front end is designed with React.js, providing an interactive and intuitive interface, while the back end, built on Node.js and Express, ensures secure data handling and smooth server operations. MongoDB is used for data storage, offering flexible schema design and efficient query capabilities. This e-commerce platform not only allows users to search and filter products by various attributes such as size, color, pattern, and brand but also provides secure payment processing via PayPal integration, enabling a seamless shopping experience.

1.1 EXISTING SYSTEM

From the requirement analysis, it is clear that while there are numerous e-commerce platforms available, many lack specialized features for men's clothing retail. Existing platforms often have limited filtering options, inadequate product categorization specific to men's fashion, and cumbersome checkout processes. Many smaller retailers still rely on generic e-commerce solutions that don't cater to the specific needs of fashion retail, such as size variations, color options, and style categories. Additionally, some platforms lack integrated payment systems and comprehensive admin controls for inventory management. The new Croyez Clothing e-commerce platform will provide a centralized digital marketplace specifically designed for men's fashion, with advanced search and filtering capabilities, intuitive product categorization, seamless payment processing, and robust admin tools for product management and order tracking.

1.2 SYSTEM STUDY

1.2.1 Understanding the Business Requirements

The first step in the system study is to understand the business requirements of the project. This involves identifying the business objectives, target audience, and the features and functionalities required in the website. For Croyez Clothing, this includes understanding men's fashion retail needs, customer shopping behaviors, and administrative requirements for managing an online clothing store.

1.2.2 Analyzing the Existing Systems and Processes

The next step is to analyze the existing systems and processes in place, including competitors' websites and current e-commerce trends. This will help identify any limitations or areas for improvement in the new website, such as enhanced filtering options, better product visualization, and more efficient checkout processes.

1.2.3 Identifying User Needs

The system study also involves identifying the needs of both customers and administrators who will be using the website. This can be done through surveys, focus groups, or other user research methods to understand pain points in online clothing shopping and administrative challenges in e-commerce management.

1.2.4 Developing Use Cases

Use cases are developed to illustrate how the website will function and how users will interact with it. This includes scenarios like browsing products, applying filters, adding items to a cart, completing a purchase, and for administrators, managing inventory and processing orders.

1.3 OBJECTIVE

The objective of this project is to develop a full-stack e-commerce application for men's clothing that allows customers to browse, search, and purchase products with ease. The application will enable users to filter products by categories, sizes, colors, patterns, and brands, and to securely complete transactions using PayPal integration. For administrators, the platform will provide comprehensive tools for product management, inventory control, and order processing. This app will feature a front-end interface using React.js, a back-end server using Node.js and Express to handle API requests and business logic, and MongoDB for data storage. The goal is to create a comprehensive, user-friendly e-commerce platform that enhances the shopping experience for customers and simplifies store management for administrators.

1.4 SCOPE

The scope of this Croyez Clothing project involves creating a comprehensive men's clothing e-commerce platform that allows customers to efficiently browse, search, and purchase products through a web application. Users will be able to register, log in, and manage their personal shopping experience securely. Key functionalities include browsing products by categories, filtering items based on attributes (size, color, pattern, brand), adding products to a shopping cart, writing product reviews, and completing purchases through secure PayPal integration. The administrative module will provide comprehensive CRUD operations for product management, allowing admins to add, update, delete, and view products, as well as track order status and manage inventory. MongoDB will serve as the database, offering flexible schema design for product variations and efficient querying capabilities. The user interface, built with React.js, will be fully responsive and optimized for use on both desktop and mobile devices. The back end, powered by Node.js and Express, will manage data requests and ensure smooth communication between the client and database. Additional security measures will safeguard user data and payment information. By focusing on these core functionalities, the project aims to deliver a user-friendly and scalable e-commerce application that meets essential user needs while allowing room for future enhancements.

CHAPTER 2

GENERAL DESCRIPTION

2.1 PROJECT PERSPECTIVE

- The growing e-commerce market, particularly in men's fashion, underscores the need for specialized platforms that offer intuitive shopping experiences tailored to clothing retail.
- The "Croyez Clothing" platform provides an efficient, accessible way for customers to browse, select, and purchase men's apparel with features specifically designed for fashion retail, such as detailed product filtering and size guidance.
- By offering a digital platform accessible from any device, users gain 24/7 access to a wide selection of men's clothing, allowing for convenient shopping regardless of location or time constraints.
- The administrative interface empowers store managers to efficiently manage inventory, track orders, and respond to customer needs through a comprehensive dashboard.

2.2 USER CHARACTERISTICS

Customers

Primarily individuals seeking an easy-to-use, visually appealing, and reliable platform for browsing and purchasing men's clothing. They require a seamless experience that includes clear navigation, detailed product information, and a secure checkout process. Key features for users include the ability to filter products by multiple attributes, view high-quality product images, read product reviews, and track order status. Additionally, users appreciate personalized recommendations, wishlist functionality, and a responsive design that works across devices.

Administrators

Responsible for maintaining the system's functionality, product inventory, and order fulfillment. Administrators require a comprehensive dashboard to manage products, track orders, update inventory, and monitor customer reviews. They prioritize tools for bulk product uploads, easy category management, order status updates, and sales analytics. Robust reporting

and data-tracking systems help administrators optimize the platform, manage stock levels, and identify sales trends for business improvements.

Both user and admin needs are key factors in designing an efficient and satisfying user experience on the "Croyez Clothing" platform.

2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS

2.3.1 Time

The project has strict deadlines for design, development, testing, and deployment, to ensure the timely release of the e-commerce platform. Delays in meeting these milestones could affect the platform's launch and competitiveness in the market.

2.3.2 Budget Limitations

The project may face budget constraints that restrict resources available for development, technology procurement, and marketing efforts. This may influence the selection of certain features, technologies, or third-party services integrated into the platform.

2.3.3 Security

The platform must adhere to industry standards and regulations to ensure the privacy and security of users' personal and payment information. Compliance with data security protocols (e.g., encryption, secure payment gateways) and e-commerce standards may impose specific security requirements, shaping how data is managed and transactions are processed.

2.3.4 Scalability

The architecture of the e-commerce platform must be scalable to handle potential growth in product listings, user accounts, and concurrent transactions as the business expands. Constraints may arise if the architecture does not support increased data storage or fails to perform under heavy loads during peak shopping periods.

CHAPTER 3

REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENTS

3.1.1 User Authentication and Management

Users should be able to securely register, log in, and log out to access their accounts. This allows customers to have personalized shopping experiences, save favorite items, track orders, and manage their profiles. User accounts should be protected through secure authentication methods, such as email/password and third-party sign-in (e.g., Google login).

3.1.2 Product Browsing and Filtering

The application should allow users to browse products by categories (e.g., shirts, pants, accessories) and filter items based on multiple attributes including size, color, pattern, and brand. The filtering system should be intuitive and responsive, updating results in real-time to enhance the shopping experience.

3.1.3 Shopping Cart and Checkout

Users should be able to add products to a shopping cart, modify quantities, remove items, and proceed to checkout. The checkout process should be streamlined, with clear steps for shipping information, order review, and payment selection, including integration with PayPal for secure transactions.

3.1.4 Product Rating and Reviews

Customers should have the ability to rate products and write reviews after purchase. This feature enhances the shopping experience by providing valuable feedback for other potential buyers and helps build trust in the platform.

3.1.5 Admin Product Management

Administrators should have comprehensive CRUD (Create, Read, Update, Delete) capabilities for managing product listings. This includes adding new products with detailed information, updating existing products, removing discontinued items, and viewing product performance metrics.

3.1.6 Order Management

The admin dashboard should provide tools for tracking and managing orders, including viewing order details, updating order status, and generating order reports. This ensures efficient order fulfillment and customer satisfaction.

3.2 NON-FUNCTIONAL REQUIREMENTS

3.2.1 Performance

The application should respond quickly to user inputs and load product listings efficiently. This includes optimized data retrieval from MongoDB to ensure a smooth shopping experience, even during peak usage periods. The application's performance should be optimized to handle a large volume of product listings and concurrent users without degradation.

3.2.2 Usability

The application should have an intuitive, user-friendly interface with easy navigation and clearly labeled sections. The design should focus on simplicity, allowing users to find products, apply filters, and complete purchases with minimal effort. User interfaces should follow e-commerce usability principles to enhance customer satisfaction and conversion rates.

3.2.3 Reliability

The application should be reliable and stable, with mechanisms for error handling and data recovery to prevent data loss or transaction failures. This is especially important during checkout processes, where reliability directly impacts customer trust and satisfaction.

3.2.4 Security

The platform must implement robust security measures to protect user data, payment information, and administrative functions. This includes secure authentication, encrypted communication, protection against common web vulnerabilities, and compliance with e-commerce security standards.

3.2.5 Availability

The application should maintain high availability, with minimal downtime for maintenance and updates. Backup and failover systems should be implemented to ensure that users have uninterrupted access to the shopping platform and that order processing continues without disruption.

CHAPTER 4

DETAILED DESIGN

4.1 ARCHITECTURAL DESIGN

The architectural design includes various diagrams such as use case diagram, sequence diagram and activity diagram for both user and admin modules.

4.1.1 MODULE CHARACTERISTICS

The project contains two main modules as follows and their description is given below:

- User module
- Admin module

4.1.1.1 User Module

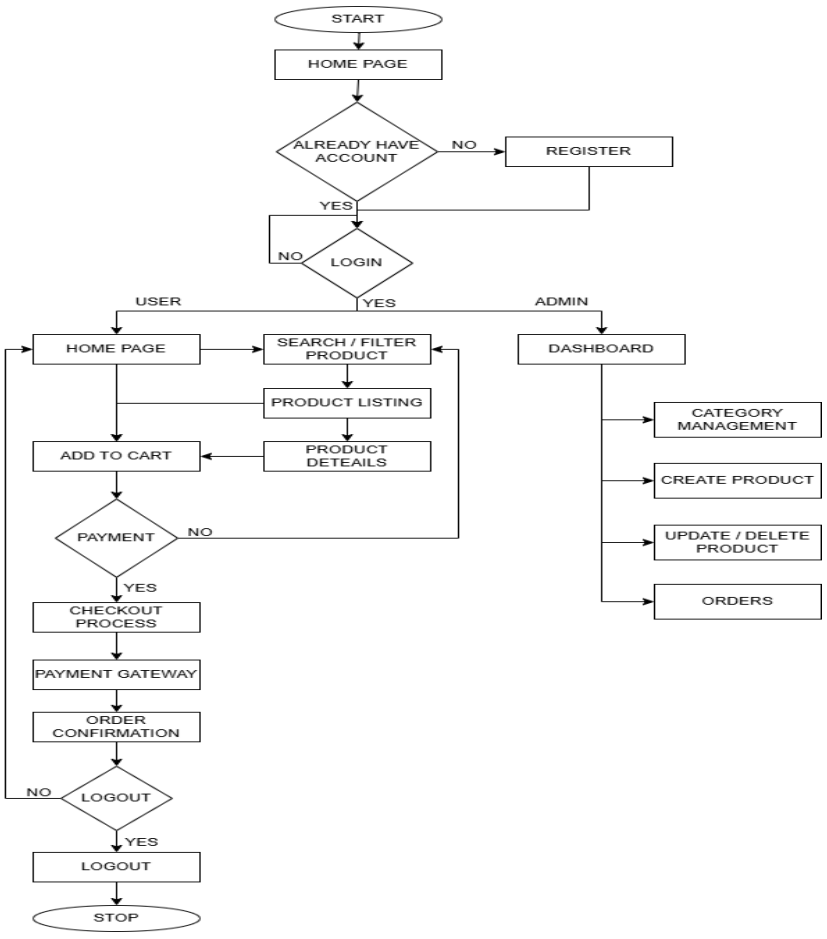
The user module encompasses the customer-facing aspects of the e-commerce platform, including:

- **Home Page**: Features promotional banners, featured products, new arrivals, and navigation to various product categories.
- **Product Catalog**: Displays products with filtering options by category, size, color, pattern, and brand.
- **Product Detail Page**: Shows comprehensive product information, including images, descriptions, available sizes, colors, pricing, and customer reviews.
- **Shopping Cart**: Allows users to add, remove, and modify quantities of selected items before checkout.
- **Checkout Process**: Guides users through shipping information, order review, and payment selection with PayPal integration.
- **User Profile**: Enables customers to manage personal information, view order history, and track current orders.
- **Wishlist**: Lets users save products for future consideration.
- **Reviews and Ratings**: Allows customers to provide feedback on purchased products.

4.1.1.2 Admin Module

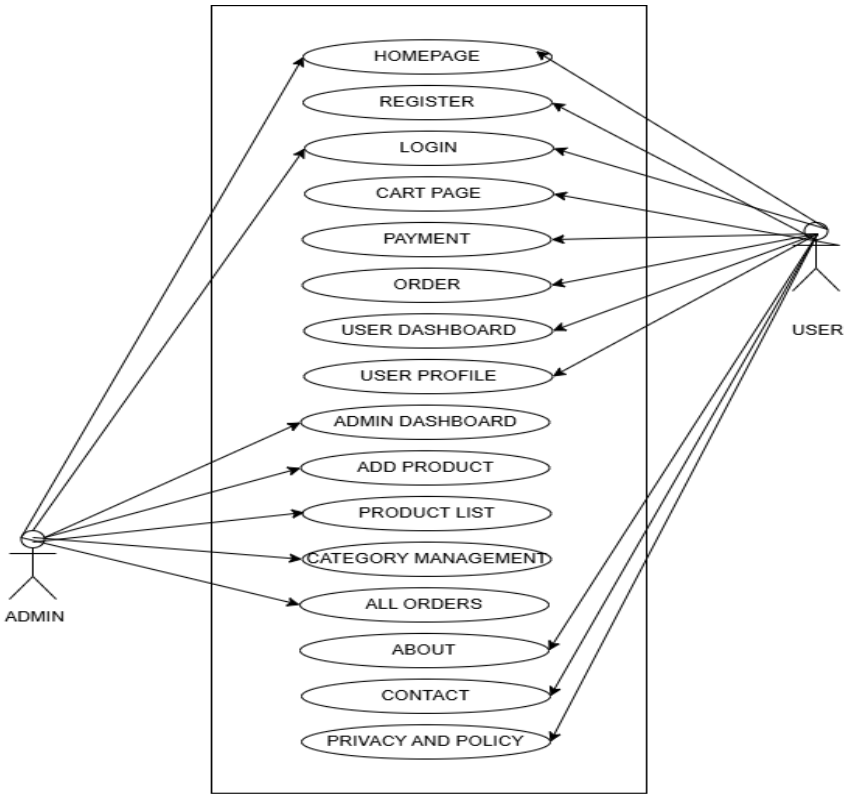
The admin module provides backend management functionalities for store administrators:

- **Dashboard**: Displays key metrics such as sales trends, popular products, inventory alerts, and recent orders.
- **Product Management**: Offers comprehensive CRUD operations for product listings, including adding new products, updating existing ones, and removing discontinued items.
- **Inventory Control**: Allows tracking of stock levels, setting low-stock alerts, and managing product availability.
- **Order Management**: Enables viewing, processing, and updating order status throughout the fulfillment cycle.
- **Customer Management**: Provides access to customer information, purchase history, and communication tools.



4.1.1.3 USE CASE DIAGRAM

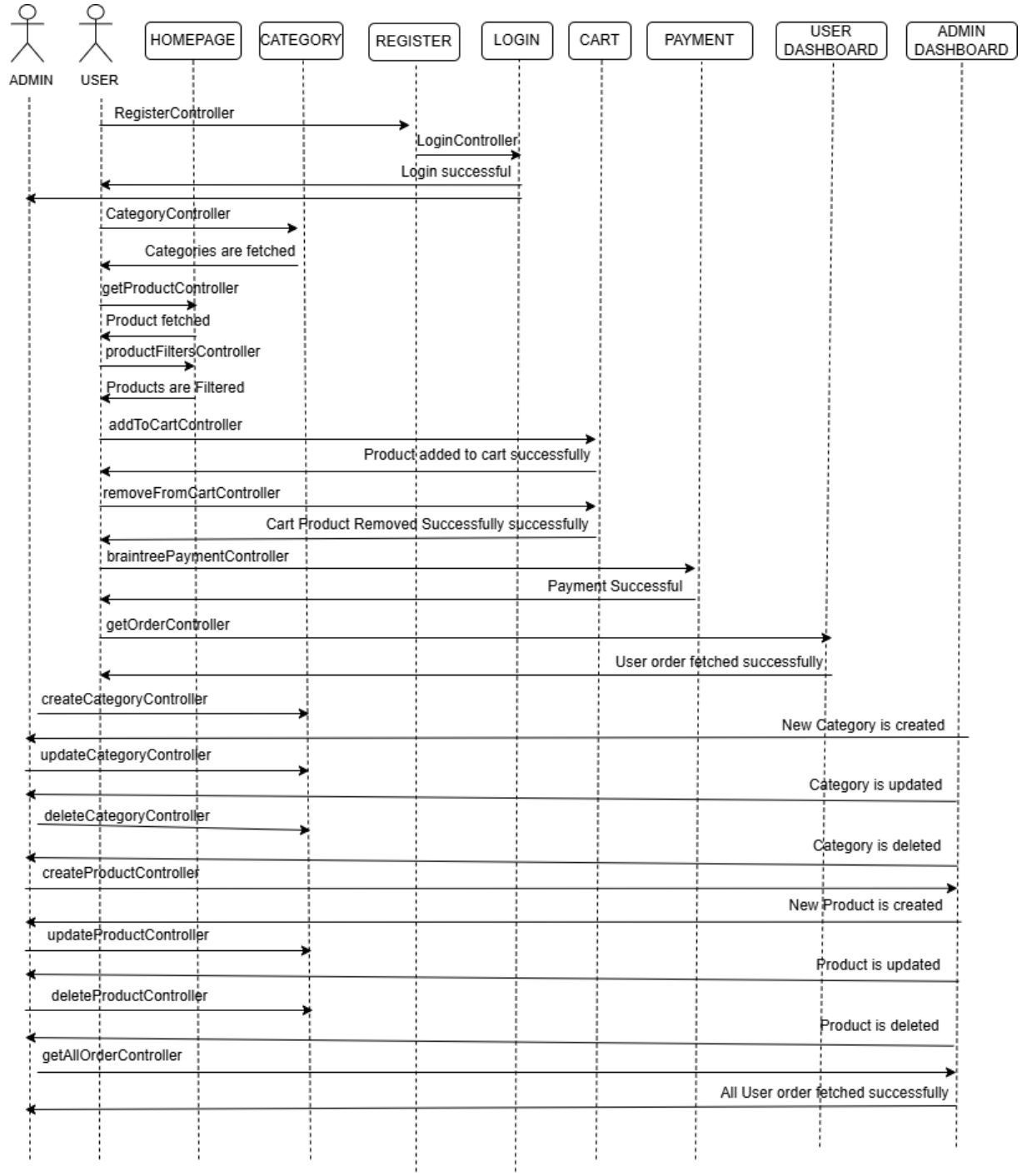
A use case diagram is a dynamic or behavior diagram in UML. Use case diagrams model the functionality of a system using actors and use cases. Use cases are sets of actions, services, and functions that the system needs to perform. A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the different use cases in which the user is involved.



[Figure 4.2: Use case diagram of Croyez Clothing]

4.1.1.4 SEQUENCE DIAGRAM

Sequence diagram is one of the five UML diagrams used to model the dynamic nature of a system. It defines the sequence of object interactions during their lifetime.



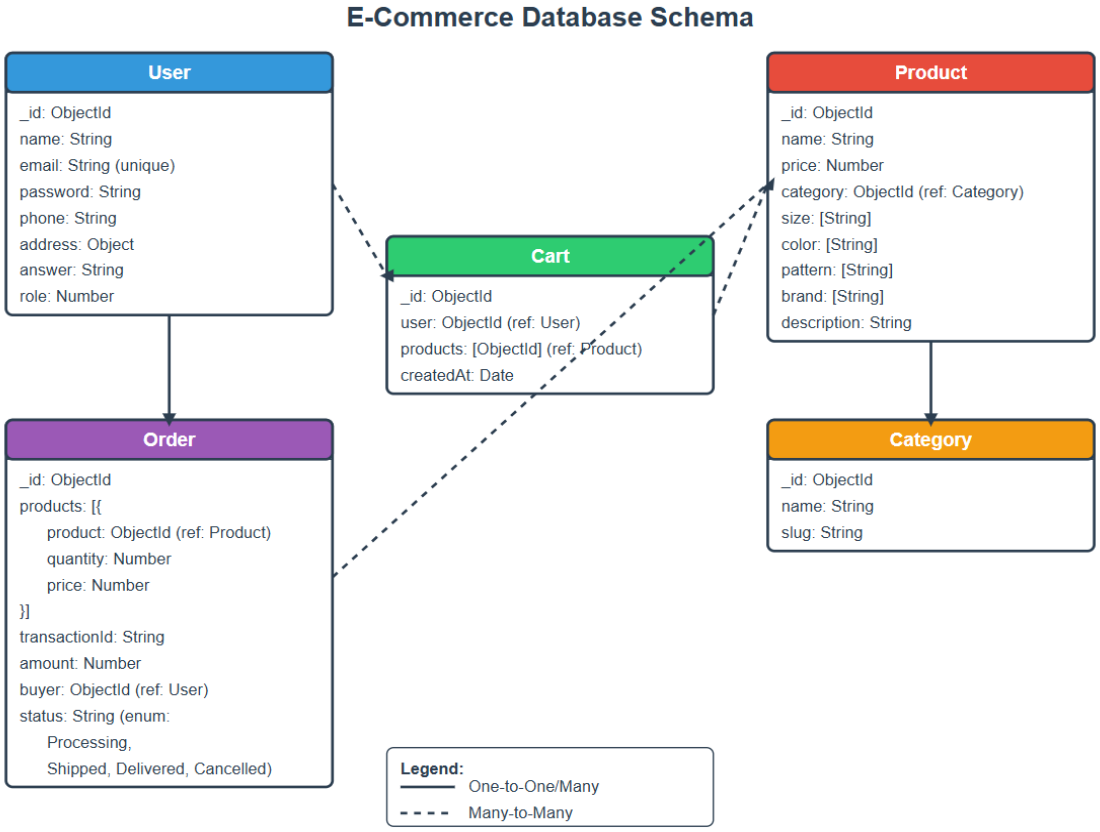
[Figure 4.3: Sequence diagram of Croyez Clothing]

4.2 INTERFACE DESIGN

The Croyez Clothing e-commerce platform provides a user-friendly interface that enhances the convenience and experience of online shopping for men's apparel. The user interface includes a Home screen that displays featured products, promotional banners, and navigation to product categories, providing users with an engaging entry point to the store. The application also includes sections such as Product Catalog with advanced filtering options, detailed Product Pages with high-quality images and sizing information, and a streamlined Shopping Cart and Checkout process. Users can access their Profile section to manage account settings, view order history, and track active orders. The administrative interface offers a comprehensive Dashboard for store management, including product creation tools, inventory control, order processing, and analytics reporting. Designed to be intuitive, visually appealing, and consistent across all screens, the interface allows both customers and administrators to interact with the platform with ease and efficiency, promoting a positive shopping experience and effective store management.

4.3 DATABASE DESIGN

The database used in this system is MongoDB, a NoSQL document database that stores data in flexible, JSON-like documents. In this e-commerce application, MongoDB provides a scalable and adaptable solution for managing product data with varying attributes and configurations. The database schema includes collections for Products, Users, Orders, Reviews, and Categories, with relationships between these entities to support the application's functionality. MongoDB's flexible schema allows for efficient storage of product variations (sizes, colors) and easily accommodates new product attributes without requiring schema modifications. The database design also supports efficient querying based on multiple filter criteria, which is essential for the advanced product filtering capabilities of the platform. This document-oriented database design offers a reliable, scalable approach for managing e-commerce data with complex relationships and varying attributes.



[Figure 4.4: Database Design of Croyez Clothing]

4.4 OUTPUT DESIGN

Output design generally refers to the results and information that are generated by the system for end-users; it should be understandable with an enhanced format. For the Croyez Clothing e-commerce platform, output design encompasses the visual presentation of product listings, detailed product information, shopping cart summaries, order confirmations, and admin reports. Each output is designed to be clear, informative, and visually appealing, with appropriate use of typography, color, and layout to highlight important information. For customers, outputs include responsive product cards that display key information at a glance, detailed product pages with comprehensive specifications, and order summaries with clear pricing and status updates. For administrators, outputs include dashboard analytics with visual representations of sales data, inventory reports highlighting stock levels, and order management interfaces that prioritize actionable information. The output is designed to be attractive, convenient, and informative, enhancing the overall user experience and operational efficiency of the platform.

CHAPTER 5

TESTING

5.1 UNIT TESTING

Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinized for proper operation. For the Croyez Clothing platform, unit testing was performed on individual components such as the product filtering mechanism, shopping cart functionality, user authentication, PayPal integration, and administrative product management tools. Each component was tested in isolation to ensure it functioned correctly before integration with other parts of the system. The React components for product display, filtering interfaces, and checkout forms were tested using Jest and React Testing Library, while backend API endpoints were tested using Mocha and Chai. This approach helped identify and resolve issues at the component level before they could affect the entire system.

5.2 REGRESSION TESTING

Regression testing helps in identifying any unintended side effects or issues that may arise due to code changes, updates to dependencies, or modifications in the underlying infrastructure. For the Croyez Clothing platform, regression testing was conducted after implementing new features or making significant changes to existing functionality. This included retesting the product filtering system after adding new filter options, verifying the checkout process after integrating PayPal, and ensuring that the administrative dashboard remained functional after adding new product management capabilities. By systematically retesting critical paths through the application, the development team ensured that new features did not adversely affect existing functionality, maintaining the integrity and reliability of the e-commerce platform.

5.3 VALIDATION TESTING

Validation testing is the process of evaluating software during or at the end of the development process to determine whether it satisfies specified business requirements. For Croyez Clothing, validation testing focused on ensuring that the application met the defined requirements for

both customer and administrative users. This included validating that customers could effectively search for products using multiple filter criteria, add items to their cart, complete purchases through PayPal, and leave product reviews. For the administrative side, validation testing verified that administrators could perform all necessary product management tasks, track orders, and access required reports. Through validation testing, the team confirmed that the e-commerce platform fulfilled its intended purpose and provided value to both customers and business operators.

5.4 VERIFICATION TESTING

Verification is the process of evaluating work-products of a development phase to determine whether they meet the specified requirements for that phase. For the Croyez Clothing project, verification testing was conducted at each development milestone to ensure that the implemented features aligned with the design specifications. This included verifying that the product database schema correctly supported the required filtering capabilities, that the user interface components rendered properly across different devices, and that the API endpoints returned the expected data formats. By verifying each component against its specifications before moving to the next development phase, the team maintained consistency and quality throughout the project lifecycle.

5.5 INTEGRATION TESTING

Integration testing for Croyez Clothing ensured smooth interaction between components like product filters and the database, the shopping cart and checkout process, and admin tools with the product catalog. Focus was placed on verifying the PayPal payment gateway integration and ensuring frontend actions correctly triggered backend operations, maintaining data consistency.

CHAPTER 6

CONCLUSION AND FUTURE WORK

The Croyez Clothing e-commerce platform provides an intuitive and efficient online shopping experience for men's apparel. By leveraging technologies like React.js for the front end, Node.js and Express for the backend, and MongoDB for data storage, this application delivers a seamless shopping experience, offering comprehensive product browsing, intuitive filtering, secure checkout, and reliable order management. Through functionalities like advanced product filtering, secure PayPal integration, product ratings, and comprehensive admin tools, users can easily find and purchase products while administrators can efficiently manage inventory and fulfill orders. The platform's responsive design and user-centered approach ensure an optimal shopping experience across devices, enhancing customer satisfaction and business operations. The development of this project also demonstrates the effectiveness of modern web technologies in creating scalable, feature-rich e-commerce solutions that meet the specific needs of fashion retail. In conclusion, the Croyez Clothing application stands as a reliable solution for online men's fashion retail, providing both customers and administrators with the tools they need for a successful e-commerce operation.

Future enhancements could include implementing a recommendation system based on user browsing history, integrating additional payment gateways, developing a mobile application, implementing a loyalty program, and expanding analytics capabilities for more detailed business insights. These potential improvements would further enhance the platform's value proposition and competitiveness in the growing e-commerce market for men's fashion.

APPENDIX 1

CODING :

FRONTEND :

App.js :

```
import { Routes, Route } from "react-router-dom";
import HomePage from "./pages/HomePage";
import About from "./pages/About";
import Contact from "./pages/Contact";
import Policy from "./pages/Policy";
import Pagenotfound from "./pages/Pagenotfound";
import Register from "./pages/Auth/Register";
import Login from "./pages/Auth/Login";
import Dashboard from "./pages/user/Dashboard";
import PrivateRoute from "./components/Routes/Private";
import ForgotPasssword from "./pages/Auth/ForgotPasssword";
import AdminRoute from "./components/Routes/AdminRoute";
import AdminDashboard from "./pages/Admin/AdminDashboard";
import CreateCategory from "./pages/Admin/CreateCategory";
import CreateProduct from "./pages/Admin/CreateProduct";
import Users from "./pages/Admin/Users";
import Orders from "./pages/user/Orders";
import Profile from "./pages/user/Profile";
import Products from "./pages/Admin/Products";
import UpdateProduct from "./pages/Admin/UpdateProduct";
import Search from "./pages/Search";
import ProductDetails from "./pages/ProductDetails";
import Categories from "./pages/Categories";
import CategoryProduct from "./pages/CategoryProduct";
import CartPage from "./pages/CartPage";
import AdminOrders from "./pages/Admin/AdminOrders";
function App() {
  return (
    <>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/product/:slug" element={<ProductDetails />} />
        <Route path="/categories" element={<Categories />} />
        <Route path="/cart" element={<CartPage />} />
        <Route path="/category/:slug" element={<CategoryProduct />} />
        <Route path="/search" element={<Search />} />
        <Route path="/dashboard" element={<PrivateRoute />}>
          <Route path="user" element={<Dashboard />} />
          <Route path="user/orders" element={<Orders />} />
          <Route path="user/profile" element={<Profile />} />
        </Route>
        <Route path="/dashboard" element={<AdminRoute />}>
          <Route path="admin" element={<AdminDashboard />} />
          <Route path="admin/create-category" element={<CreateCategory />} />
```

```
          <Route path="admin/create-product" element={<CreateProduct />} />
          <Route path="admin/product/:slug" element={<UpdateProduct />} />
          <Route path="admin/products" element={<Products />} />
          <Route path="admin/users" element={<Users />} />
          <Route path="admin/orders" element={<AdminOrders />} />
        </Route>
        <Route path="/register" element={<Register />} />
        <Route path="/forgot-password" element={<ForgotPasssword />} />
        <Route path="/login" element={<Login />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
        <Route path="/policy" element={<Policy />} />
        <Route path="*" element={<Pagenotfound />} />
      </Routes>
    </>
  );
}
export default App;
```

index.js :

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";
import reportWebVitals from "./reportWebVitals";
import { BrowserRouter } from "react-router-dom";
import { AuthProvider } from "./context/auth";
import { SearchProvider } from "./context/search";
import { CartProvider } from "./context/cart";
import "antd/dist/reset.css";
const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(
  <AuthProvider>
    <SearchProvider>
      <CartProvider>
        <BrowserRouter>
          <App />
        </BrowserRouter>
      </CartProvider>
    </SearchProvider>
  </AuthProvider>
);
reportWebVitals();
```

Homepage.js :

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { Checkbox, Radio } from "antd";
import { Prices } from "../components/Prices";
import { useCart } from "../context/cart";
```

```
import axios from "axios";
const SIZES = {
  shirts: ["S", "M", "L", "XL"],
  pants: ["28", "30", "32", "34", "36"],
  tshirts: ["S", "M", "L", "XL","XXL","XXXL"],
};
const COLOR = {
  shirts: ["Red", "Blue", "Green", "Black", "White",],
  pants: ["Blue", "Black", "Brown", "Grey", "White","Sandal"],
  tshirts: ["Red", "Blue", "Green", "Black", "White","Yellow","Pink"],
};
const HomePage = () => {
  const navigate = useNavigate();
  const [cart, setCart] = useCart();
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const getAllCategory = async () => {
    try {
      const { data } = await axios.get(`${process.env.REACT_APP_API}/api/v1/category/get-
    category`, {headers: {Authorization: `Bearer ${auth.token}`,},});
      if (data?.success) { setCategories(data?.category); }
    } catch (error) { console.log(error);}
  };
  // Get all products (initial load)
  const getAllProducts = async () => {
    try {
      setLoading(true);
      const { data } = await
    axios.get(`${process.env.REACT_APP_API}/api/v1/product/product-list/${page}`);
      setLoading(false);
      setProducts(data.products);
    } catch (error) {
      setLoading(false);
      console.log(error);
    }
  };
 // Handle category filter (updated for radio buttons)
  const handleFilter = (e, id, name) => {
    e.preventDefault();
    setChecked([id]); // Only one category can be selected at a time
    // Determine if the selected category is shirts or pants
    if (name.toLowerCase().includes("tshirt")) {
      setData("tshirts");
    } else if (name.toLowerCase().includes("pant")) {
      setData("pants");
    } else if (name.toLowerCase().includes("shirt")) {
      setData("shirts");
    } else if (name.toLowerCase().includes("short")) {
  };
  // Handle size filter
  const handleSizeFilter = (e, size) => {
```

```
  e.preventDefault();
  let updatedSizes = [...sizes];
// Get filtered products
const filterProduct = async () => {
  try {
    const { data } = await
   axios.post(`${process.env.REACT_APP_API}/api/v1/product/product-filters`, {
      checked, sizes, colors, patterns, brands, radio,
    });
    setProducts(data?.products);
  } catch (error) {console.log("Error filtering products:", error);}
};
// In your HomePage.js or wherever you're adding to cart
const HandleAddToCart = async (product) => {
const { data } = await axios.post(
      `${process.env.REACT_APP_API}/api/v1/user/cart/add`,
      { product: product._id, quantity: 1 },
      {
        headers: {
          Authorization: `Bearer ${auth.token}`,
        },
      }
    );
    if (data?.success) {
      setCart(data.cart);
      toast.success(`${product.name} added to cart`);
    }
  } catch (error) {
    console.error("Error adding to cart:", error);
    if (error.response?.status === 401) {
      navigate("/login");
    } else {
      toast.error("Failed to add item to cart");
    }
  }
};
return (
  <Layout title={"All Products - Best offers"}>
    {/* Banner Image */}
    <img
      src="/images/croyez.jpg"
      className="banner-img"
      alt="bannerimage"
      width={"100%"}
      height={"300px"}
    />
    <div className="container-fluid row mt-3 home-page">
      {/* Sidebar Filters */}
      <div className="col-md-3 filters">
        {/* Category Filter */}
           {categories?.map((c) => (
```

```
            <Radio key={c._id} value={c._id}>
              {c.name}
            </Radio>
          ))}
        </Radio.Group>
      </div>
      {/* Size Filter */}
      <h4 className="text-center mt-4">Filter By Size</h4>
      <div className="d-flex flex-column">
        {SIZES[data]?.map((size) => (
          <Checkbox
            key={size}
            onChange={(e) => handleSizeFilter(e, size)}
            checked={sizes.includes(size)}
          >
            {size}
          </Checkbox>
        ))}
      </div>
      {/* Price Filter */}
      <h4 className="text-center mt-4">Filter By Price</h4>
      <div className="d-flex flex-column">
        <Radio.Group onChange={handlePriceFilter}>
          {Prices?.map((p) => (
            <div key={p._id}>
              <Radio value={p.array}>{p.name}</Radio>
            </div>
          ))}
        </Radio.Group>
      </div>
      {/* Reset Filters Button */}
      <div className="d-flex flex-column">
        <button>
          RESET FILTERS
        </button>
      </div>
    </div>
    {/* Products Section */}
    <div className="col-md-9">
      <h1 className="text-center">All Products</h1>
      <div className="d-flex flex-wrap">
        {products?.map((p) => (
          <div className="card m-2" key={p._id}>
            <img
              src={`${process.env.REACT_APP_API}/api/v1/product/product-
photo/${p._id}`}
              className="card-img-top"
              alt={p.name}/>
            <div className="card-body">
              <h5 className="card-title">{p.name}</h5>
              <h5 className="card-title card-price">
```

```
                {p.price.toLocaleString("en-US", {
                 style: "currency",
                 currency: "INR",
                })}
              </h5>
            </div>
            <p className="card-text">
              {p.description.substring(0, 60)}...
            </p>
            <div className="card-name-price">
              <button
                className="btn btn-info ms-1"
                onClick={() => navigate(`/product/${p.slug}`)}
              >
                More Details
              </button>
              <button
      className="btn btn-dark ms-1"
      onClick={() => HandleAddToCart(p)}
    >
      ADD TO CART
    </button>
            </div></div> </div>
          ))}
        </div>
      </div>
    </div>
  </Layout>
 );
};
export default HomePage;
```

CartPage.js :

```
import React, { useState, useEffect } from "react";
import Layout from "./../components/Layout/Layout";
import { useCart } from "../context/cart";
import { useAuth } from "../context/auth";
import axios from "axios";
import toast from "react-hot-toast";
import "../styles/CartStyles.css";
const CartPage = () => {
  const [auth, setAuth] = useAuth();
  const [cart, setCart] = useCart();
  const [clientToken, setClientToken] = useState("");
  const navigate = useNavigate();
  //total price
  const totalPrice = () => {
    try {
      let total = 0;
      cart?.forEach((item) => {
```

```
      total += item.product.price * item.quantity;
    });
    return total.toLocaleString("en-US", {
      style: "currency",
      currency: "INR",
    });
  } catch (error) {
    console.log(error);
  }
};
const removeCartItem = async (pid) => {
  try {
    const { data } = await
   axios.delete(`${process.env.REACT_APP_API}/api/v1/user/cart/remove/${pid}`, {
      headers: {
        Authorization: `Bearer ${auth.token}`
      }
    });
} catch (error) {
    console.error("Remove item error:", error.response?.data || error.message);
    toast.error(error.response?.data?.message || "Failed to remove item");
  }
};
//handle payments
const handlePayment = async () => {
  try {
    setLoading(true);
        // 1. Get payment method nonce
    const { nonce } = await instance.requestPaymentMethod();
    if (!nonce) {
      throw new Error("No payment nonce received");
    }
    console.log("Received nonce:", nonce);
//  Submit payment
    const { data } = await
   axios.post(`${process.env.REACT_APP_API}/api/v1/product/braintree/payment`,
      paymentData,
      {
        headers: {
          Authorization: `Bearer ${auth.token}`,
          "Content-Type": "application/json"
        }
      }
    );
    } catch (error) {
    console.error("Payment error details:", {
      nonceError: error.message,
      response: error.response?.data
    });
    toast.error(error.response?.data?.message || "Invalid payment information");
  } finally {
```

```
    setLoading(false);
  }
};
return (
  <Layout>
    <div className=" cart-page">
      <div className="container ">
        <div className="row ">
          <div className="col-md-7  p-0 m-0">
          {cart?.map((item) => (
        <div className="row card flex-row" key={item.product._id}>
          <div className="col-md-4">
            <img src={`${process.env.REACT_APP_API}/api/v1/product/product-
  photo/${item.product._id}`}
              className="card-img-top"
              alt={item.product.name}
              width="100%"
              height={"130px"}
            />
          </div>
          <div className="col-md-4">
            <p>{item.product.name}</p>
            <p>{item.product.description.substring(0, 30)}</p>
            <p>Price: {item.product.price}</p>
            <p>Quantity: {item.quantity}</p>
          </div>
          <div className="col-md-4 cart-remove-btn">
            <button
              className="btn btn-danger"
              onClick={() => removeCartItem(item.product._id)} >
              Remove
            </button>
              </div>
            </div>
          ))}
        </div>
        <div className="col-md-5 cart-summary ">
          <h2>Cart Summary</h2>
          <p>Total | Checkout | Payment</p>
          <hr />
          <h4>Total : {totalPrice()} </h4>
          {auth?.user?.address ? (
            <>
              <div className="mb-3">
                <h4>Current Address</h4>
                <h5>{auth?.user?.address}</h5>
                <button
                  className="btn btn-outline-warning"
                  onClick={() => navigate("/dashboard/user/profile")}
                >
                  Update Address
```

```
              </button>
            </div>
          </>
        ) : (
          <div className="mb-3">
            {auth?.token ? (
              <button
                className="btn btn-outline-warning"
                onClick={() => navigate("/dashboard/user/profile")}
              >
                Update Address
              </button>
            ) : (
              <button
                className="btn btn-outline-warning"
                onClick={() =>
                  navigate("/login", {
                    state: "/cart",
                  })
                }
              >
                Plase Login to checkout
              </button>
            )}
          </div>
        )}<button
              className="btn btn-primary"
              onClick={handlePayment}
              disabled={loading || !instance || !auth?.user?.address}
            >
              {loading ? "Processing ...." : "Make Payment"}
            </button>
          </>
        )}
      </div> </div></div></div></div>
  </Layout>
 );
};
export default CartPage;
```

Order.js :

```
import React, { useState, useEffect } from "react";
import UserMenu from "../../components/Layout/UserMenu";
import Layout from "./../../components/Layout/Layout";
import axios from "axios";
import { useAuth } from "../../context/auth";
import moment from "moment";
const Orders = () => {
  const [orders, setOrders] = useState([]);
  const [auth, setAuth] = useAuth();
```

```
const getOrders = async () => {
  try {
    console.log("auth?.token", auth?.token);
    const { data } = await axios.get(`${process.env.REACT_APP_API}/api/v1/auth/orders`,
  {
      headers: {
        Authorization: `Bearer ${auth?.token}`,
      },
    });
    setOrders(data?.orders || []);
  } catch (error) {
    console.error("Error fetching orders:", error);
  }
};
return (
  <Layout title={"Your Orders"}>
    <div className="container-flui p-3 m-3 dashboard">
      <div className="row">
        <div className="col-md-3">
          <UserMenu />
        </div>
        <div className="col-md-9">
          <h1 className="text-center">All Orders</h1>
          {orders?.map((o, i) => {
            return (
              <div className="border shadow">
                <table className="table">
                  <thead>
                    <tr>
                      <th scope="col">NO</th>
                      <th scope="col">Status</th>
                      <th scope="col">Buyer</th>
                      <th scope="col">Address</th>
                      <th scope="col">Date</th>
                      <th scope="col">Payment</th>
                      <th scope="col">Price</th>
                    </tr>
                  </thead>
                  <tbody>
                    <tr>
                      <td>{i + 1}</td>
                      <td>{o?.status}</td>
                      <td>{o?.buyer?.name}</td>
                      <td>{o?.buyer?.address}</td>
                      <td>{moment(o?.createdAt).fromNow()}</td>
                      <td>{o?.payment?.success ? "Failed":"Success"}</td>
                      <td>{o?.amount}</td>
                    </tr>
                  </tbody>
                </table>
                <div className="container">
```

```
                {o?.products?.map((p, i) => (
                  <div className="row mb-2 p-3 card flex-row" key={p._id}>
                    <div className="col-md-4">
                      <img
                        src={`${process.env.REACT_APP_API}/api/v1/product/product-
    photo/${p.product}`}
                        className="card-img-top"
                        alt={p.name}
                        width="100px"
                        height={"100px"}
                      />
                    </div>
                    <div className="col-md-8">
                      <p>{p.name}</p>
                      <p>Price : {p.price}</p>
                      <p>Quantity : {p.quantity}</p>
                    </div>
                  </div>
                ))}
              </div>
            </div>
          );
        })}
      </div>
    </div>
  </div>
  </Layout>
 );
};

export default Orders;
```

AdminOrders.js :

```
import React, { useState, useEffect } from "react";
import axios from "axios";
import toast from "react-hot-toast";
import AdminMenu from "../../components/Layout/AdminMenu";
import Layout from "../../components/Layout/Layout";
import { useAuth } from "../../context/auth";
const AdminOrders = () => {
  const [status, setStatus] = useState([
    "Not Process",
    "Processing",
    "Shipped",
    "deliverd",
    "cancel",
  ]);
  const [changeStatus, setCHangeStatus] = useState("");
  const [orders, setOrders] = useState([]);
  const [auth, setAuth] = useAuth();
```

```
  const getOrders = async () => {
    try {
      console.log("auth?.token", auth?.token);
      const { data } = await axios.get(`${process.env.REACT_APP_API}/api/v1/auth/all-
orders`, {
        headers: {
          Authorization: `Bearer ${auth?.token}`,
        },
      });
      setOrders(data?.orders || []);
    } catch (error) {
      console.error("Error fetching orders:", error);
    }
  };
  useEffect(() => {
    if (auth?.token) getOrders();
  }, [auth?.token]);
    return (
    <Layout title={"Your Orders"}>
    <div className="container-flui p-3 m-3 dashboard">
      <div className="row">
        <div className="col-md-3">
          <AdminMenu />
        </div>
        <div className="col-md-9">
          <h1 className="text-center">All Customer Orders</h1>
          {orders?.map((o, i) => {
            return (
              <div className="border shadow">
                <table className="table">
                  <thead>
                    <tr>
                      <th scope="col">#</th>
                      <th scope="col">Status</th>
                      <th scope="col">Buyer</th>
                      <th scope="col">Address</th>
                      <th scope="col">Date</th>
                      <th scope="col">Payment</th>
                      <th scope="col">Price</th>
                    </tr>
                  </thead>
                  <tbody>
                    <tr>
                      <td>{i + 1}</td>
                      <td>{o?.status}</td>
                      <td>{o?.buyer?.name}</td>
                      <td>{o?.buyer?.address}</td>
                      <td>{moment(o?.createdAt).fromNow()}</td>
                      <td>{o?.payment?.success ? "Failed":"Success"}</td>
                      <td>{o?.amount}</td>
                    </tr>
```

```
            </tbody>
          </table>
          <div className="container">
            {o?.products?.map((p, i) => (
              <div className="row mb-2 p-3 card flex-row" key={p._id}>
                <div className="col-md-4">
                  <img
                    src={`${process.env.REACT_APP_API}/api/v1/product/product-
photo/${p.product}`}
                    className="card-img-top"
                    alt={p.name}
                    width="100px"
                    height={"100px"}
                  />
                </div>
                <div className="col-md-8">
                  <p>{p.name}</p>
                  <p>Price : {p.price}</p>
                  <p>Quantity : {p.quantity}</p>
                </div>
              </div>
            ))}
          </div>
        </div>
      );
    })}
  </div>
 </div>
</div>
</Layout>
);
};
export default AdminOrders;
```

BACKEND :

Server.js :

```
import express from "express";
import colors from "colors";
import dotenv from "dotenv";
import morgan from "morgan";
import connectDB from "./config/db.js";
import authRoutes from "./routes/authRoute.js";
import categoryRoutes from "./routes/categoryRoutes.js";
import productRoutes from "./routes/productRoutes.js";
import userRoutes from "./routes/userRoutes.js";
import cors from "cors";
//configure env
dotenv.config();
//databse config
```

```
connectDB();
//rest object
const app = express();
//middelwares
app.use(cors());
app.use(express.json());
app.use(morgan("dev"));
//routes
app.use("/api/v1/auth", authRoutes);
app.use("/api/v1/category", categoryRoutes);
app.use("/api/v1/product", productRoutes);
app.use("/api/v1/user", userRoutes); // Add this line to mount user routes
//rest api
app.get("/", (req, res) => {
  res.send("<h1>Welcome to ecommerce app</h1>");
});
//PORT
const PORT = process.env.PORT || 8080;

//run listen
app.listen(PORT, () => {
  console.log(
    `Server Running on ${process.env.DEV_MODE} mode on port ${PORT}`.bgCyan
      .white
  );
});
```

db.js :

```
import mongoose from "mongoose";
import colors from "colors";
import productModel from "../models/productModel.js";
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URL);
    console.log(
      `Conneted To Mongodb Databse ${conn.connection.host}`.bgMagenta.white
    );
  } catch (error) {
    console.log(`Errro in Mongodb ${error}`.bgRed.white);
  }
};

export default connectDB;
```

ROUTER :

authRoute.js :

```
import express from "express";
import {
```

```
  registerController,
  loginController,
  testController,
  forgotPasswordController,
  updateProfileController,
  getOrdersController,
  getAllOrdersController,
  orderStatusController,
} from "../controllers/authController.js";
import { isAdmin, requireSignIn } from "../middlewares/authMiddleware.js";
const router = express.Router();
//REGISTER || METHOD POST
router.post("/register", registerController);
//LOGIN || POST
router.post("/login", loginController);
//Forgot Password || POST
router.post("/forgot-password", forgotPasswordController);
//test routes
router.get("/test", requireSignIn, isAdmin, testController);
//protected User route auth
router.get("/user-auth", requireSignIn, (req, res) => {
  res.status(200).send({ ok: true });
});
//protected Admin route auth
router.get("/admin-auth", requireSignIn, isAdmin, (req, res) => {
  res.status(200).send({ ok: true });
});
//update profile
router.put("/profile", requireSignIn, updateProfileController);
//orders
router.get("/orders", requireSignIn, getOrdersController);
//all orders
router.get("/all-orders", requireSignIn, isAdmin, getAllOrdersController);
// order status update
router.put(
  "/order-status/:orderId",
  requireSignIn,
  isAdmin,
  orderStatusController
);
export default router;
```

CategoryRoutes.js :

```
import express from "express";
import { isAdmin, requireSignIn } from "./../middlewares/authMiddleware.js";
import {
  categoryControlller,
  createCategoryController,
  deleteCategoryCOntroller,
  singleCategoryController,
```

```
  updateCategoryController,
} from "./../controllers/categoryController.js";
const router = express.Router();
//routes
// create category
router.post(
  "/create-category", requireSignIn,  isAdmin,  createCategoryController);
//update category
router.put(
  "/update-category/:id",  requireSignIn,  isAdmin,  updateCategoryController);
//getALl category
router.get("/get-category", categoryControllller);
//single category
router.get("/single-category/:slug", singleCategoryController);
//delete category
router.delete(
  "/delete-category/:id",  requireSignIn,  isAdmin,  deleteCategoryCOntroller);
export default router;
```

ProductRoutes.js :

```
import express from "express";
import {
  braintreePaymentController,
  braintreeTokenController,
  createProductController,
  deleteProductController,
  getProductController,
  getSingleProductController,
  productCategoryController,
  productCountController,
  productFiltersController,
  productListController,
  productPhotoController,
  realtedProductController,
  searchProductController,
  updateProductController,
} from "../controllers/productController.js";
import { isAdmin, requireSignIn } from "../middlewares/authMiddleware.js";
import formidable from "express-formidable";
const router = express.Router();
router.post( "/create-product",
requireSignIn,  isAdmin,  formidable(),  createProductController);
//routes
router.put(
  "/update-product/:pid",
  requireSignIn,
  isAdmin,
  formidable(),
  updateProductController
);
```

```
//get products
router.get("/get-product", getProductController);
//single product
router.get("/get-product/:slug", getSingleProductController);
//get photo
router.get("/product-photo/:pid", productPhotoController);
//delete rproduct
router.delete("/delete-product/:pid", deleteProductController);
//filter product
router.post("/product-filters", productFiltersController);
//product count
router.get("/product-count", productCountController);
//product per page
router.get("/product-list/:page", productListController);
//search product
router.get("/search/:keyword", searchProductController);
//similar product
router.get("/related-product/:pid/:cid", realtedProductController);
//category wise product
router.get("/product-category/:slug", productCategoryController);
//payments routes
//token
router.get("/braintree/token", braintreeTokenController);
//payments
router.post("/braintree/payment", requireSignIn, braintreePaymentController);
export default router;
```

UserRoutes.js :

```
import express from "express";
import { requireSignIn } from "../middlewares/authMiddleware.js";
import {
  getCartController,
  updateCartController,
  mergeCartController,
  addToCartController,
  removeFromCartController,
  clearCartController
} from "../controllers/cartController.js";
const router = express.Router();
// GET cart - Get user's cart
router.get("/cart", requireSignIn, getCartController);
// POST cart - Update entire cart (replace)
router.post("/cart",   requireSignIn,  updateCartController);
// POST cart/add - Add single item to cart
router.post("/cart/add",   requireSignIn,  addToCartController);
// DELETE cart/remove/:productId - Remove item from cart
router.delete("/cart/remove/:productId",  requireSignIn,  removeFromCartController);
// POST cart/merge - Merge guest cart with user cart
router.post("/cart/merge", requireSignIn,  mergeCartController);
// DELETE cart/clear - Clear entire cart
```

```
router.delete("/cart/clear",  requireSignIn,  clearCartController);
// In userRoutes.js
router.get("/debug-cart/:userId", requireSignIn, async (req, res) => {
  try {
    // Verify the requesting user matches the debug user
    if (req.user._id.toString() !== req.params.userId) {
      return res.status(403).json({ error: "Unauthorized" });
    }
 const user = await userModel.findById(req.params.userId)
      .populate("cart.product", "name price") // Only get needed fields
      .lean();
    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }
    res.json({
      userId: user._id,
      cartCount: user.cart.length,
      cartItems: user.cart
    });
  } catch (error) {
    console.error("Debug error:", error);
    res.status(500).json({
      error: "Server error",
      details: error.message
    });
  }
});
export default router;
```

APPENDIX 2

SNAPSHOT

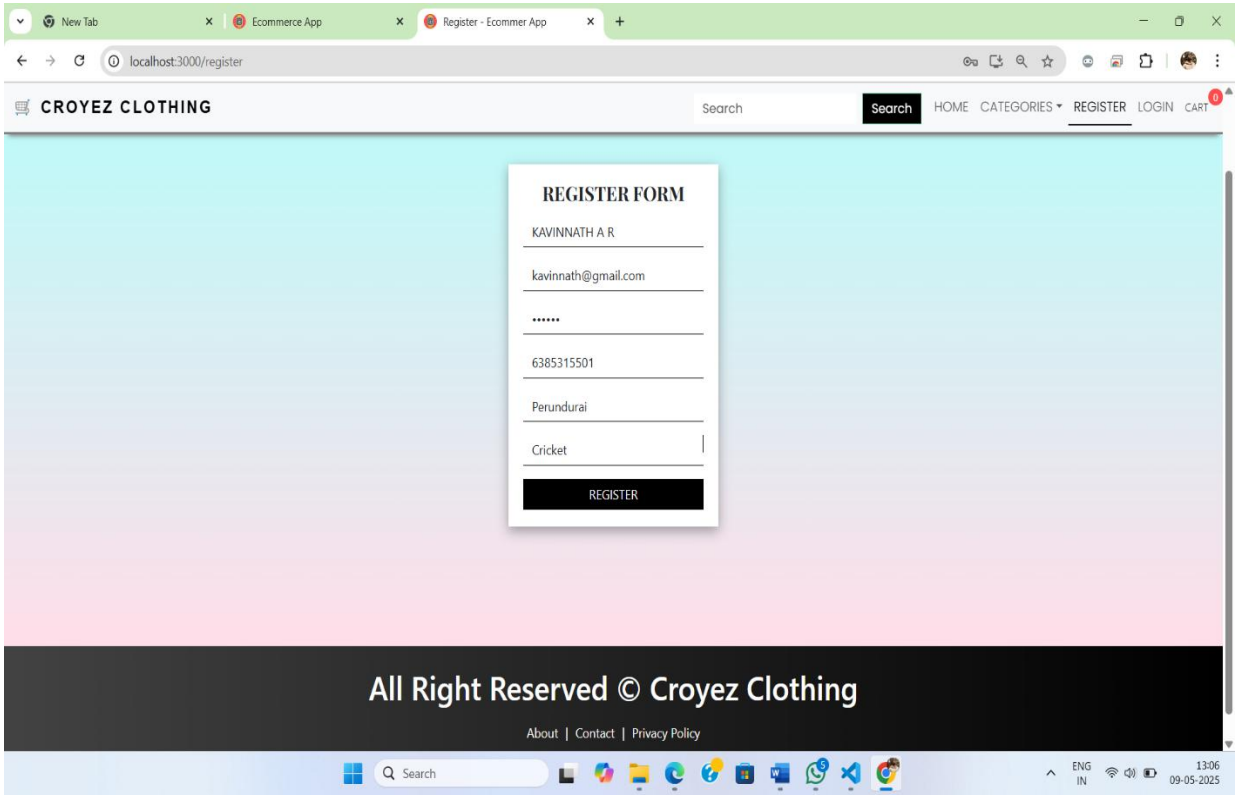


Figure A2.1 REGISTER

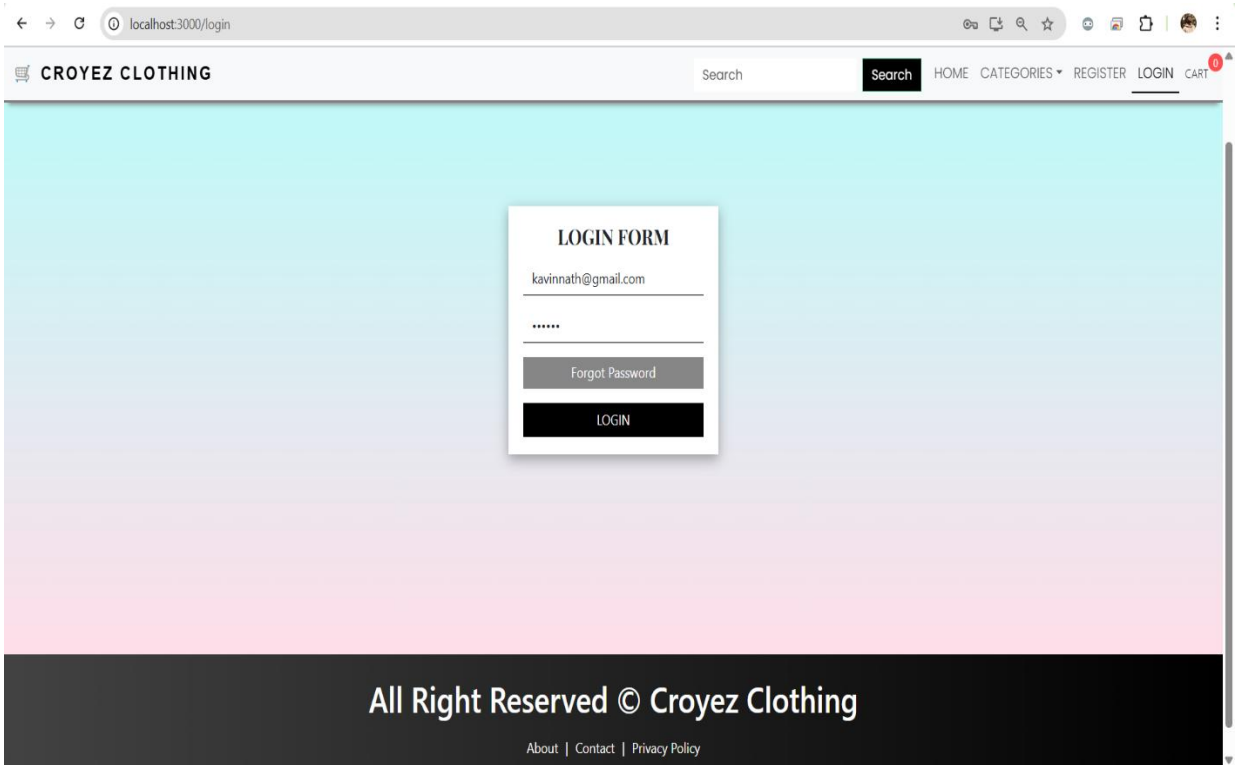


Figure A2.2 LOGIN

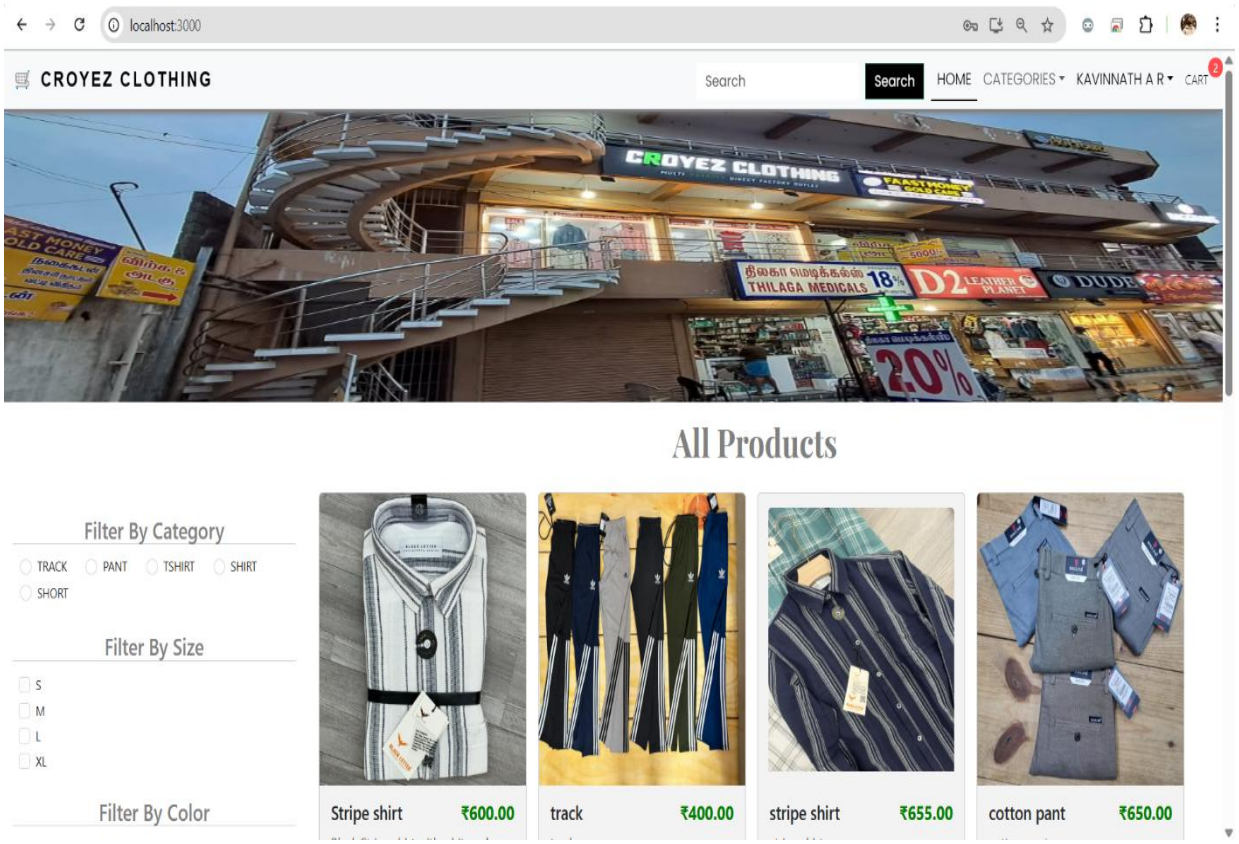


Figure A2.3 HOMEPAGE

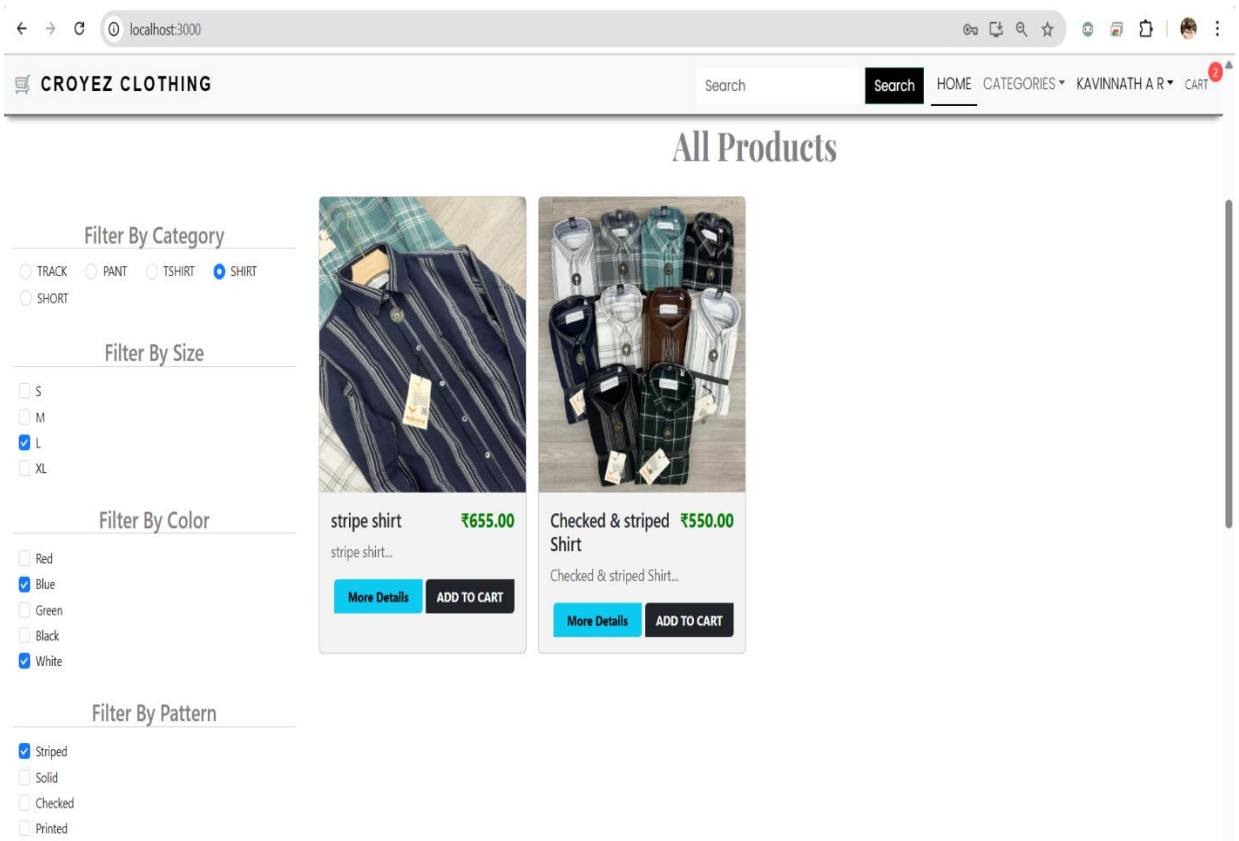


Figure A2.4 HOMEPAGE FILTERATION

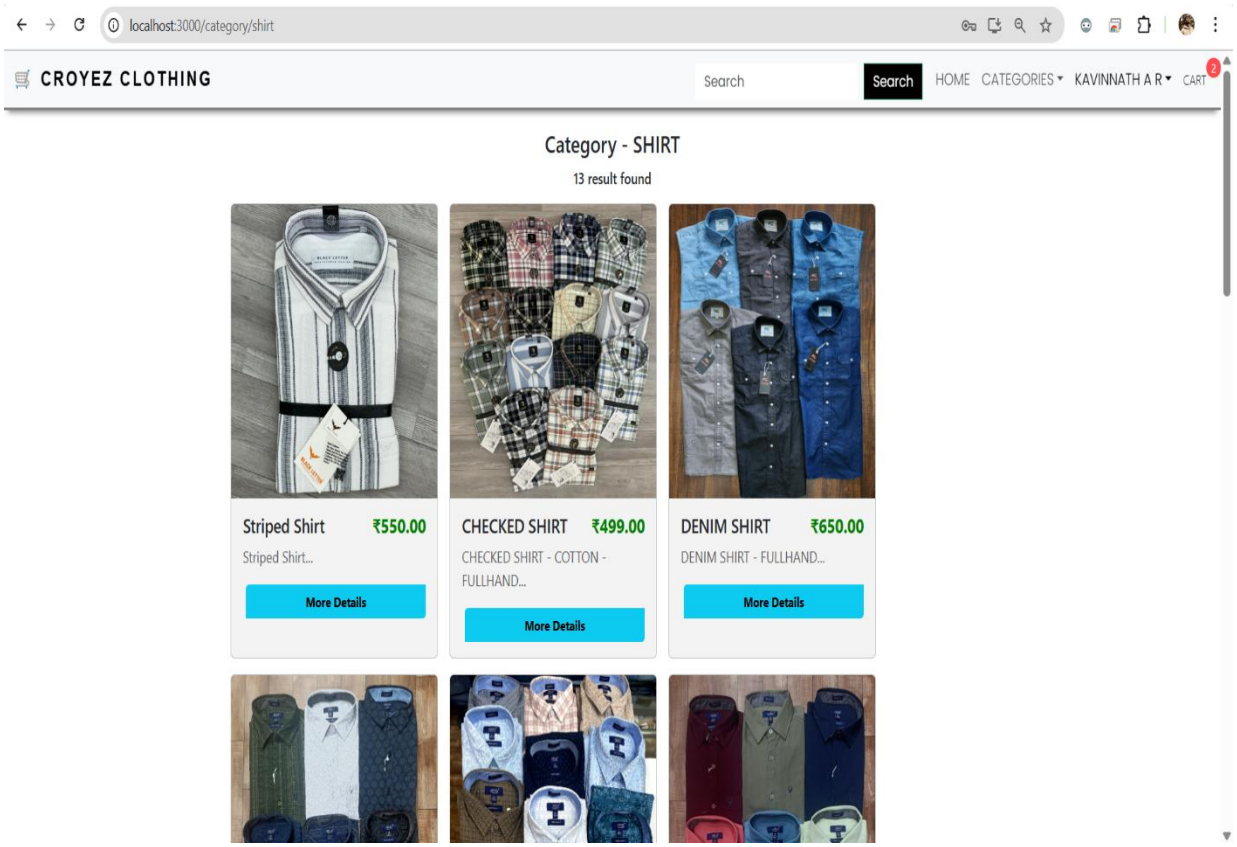


Figure A2.5 CATEGORY

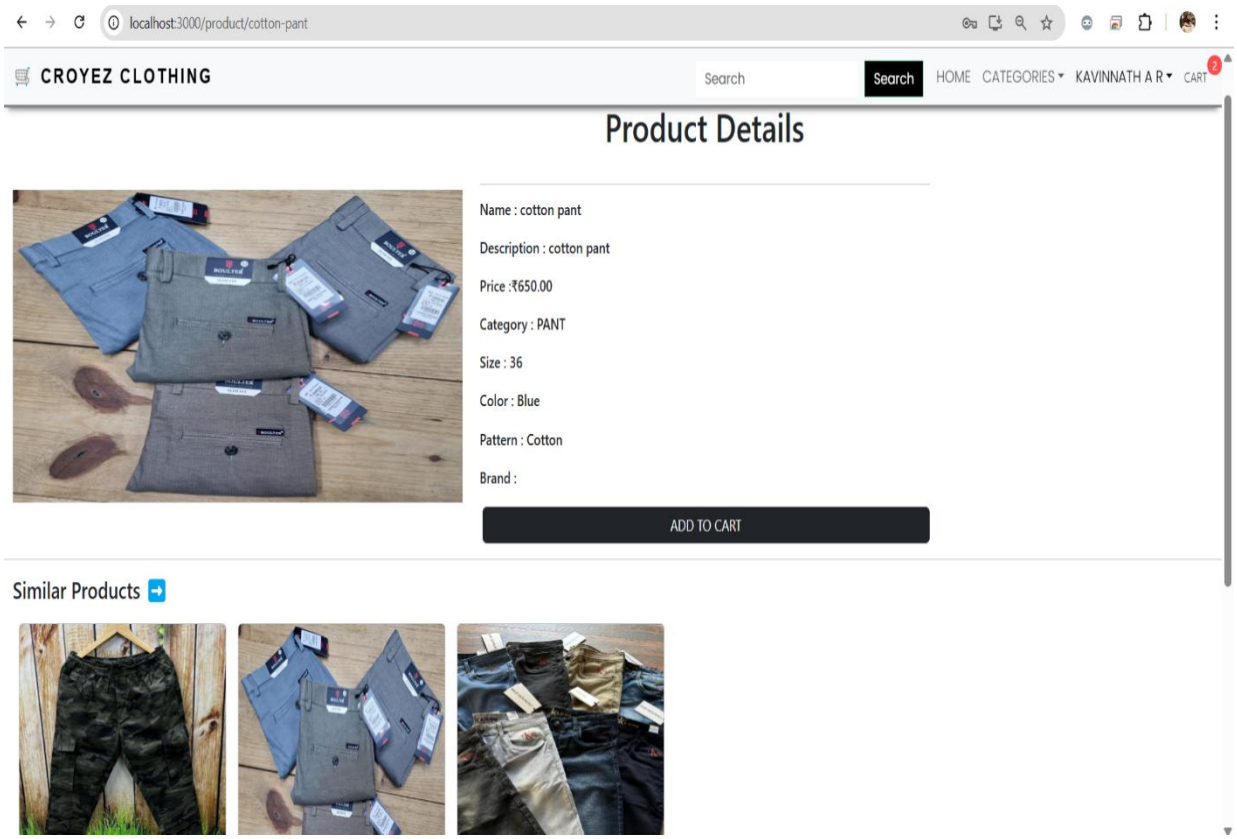


Figure A2.6 PRODUCT DETAILS

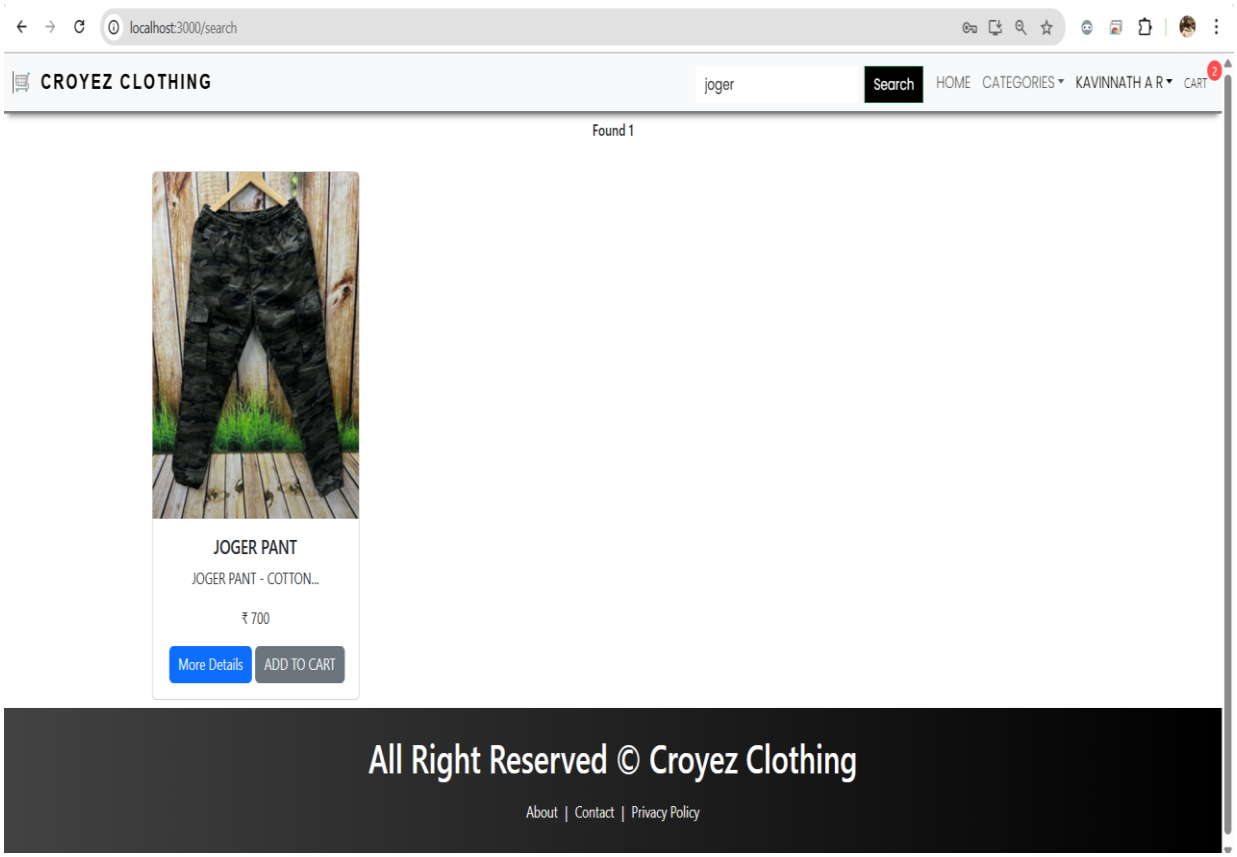


Figure A2.7 SEARCH

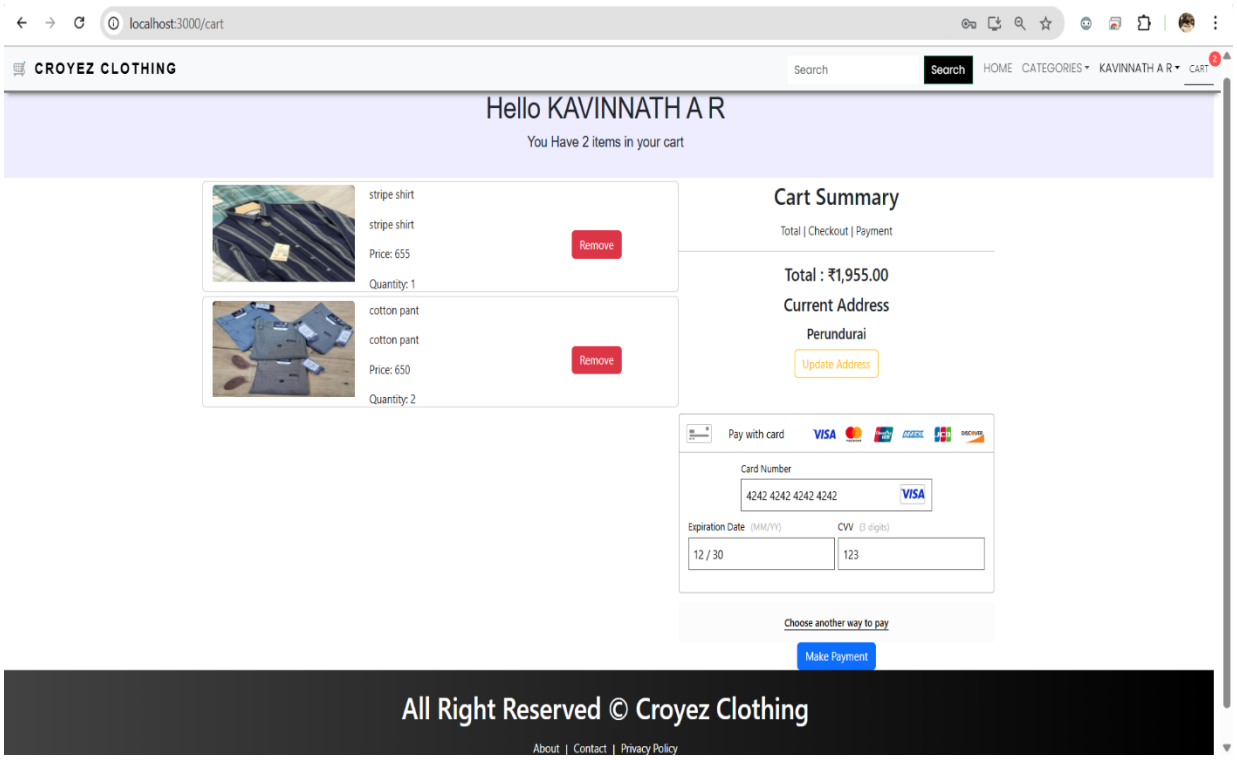


Figure A2.8 ADD TO CART

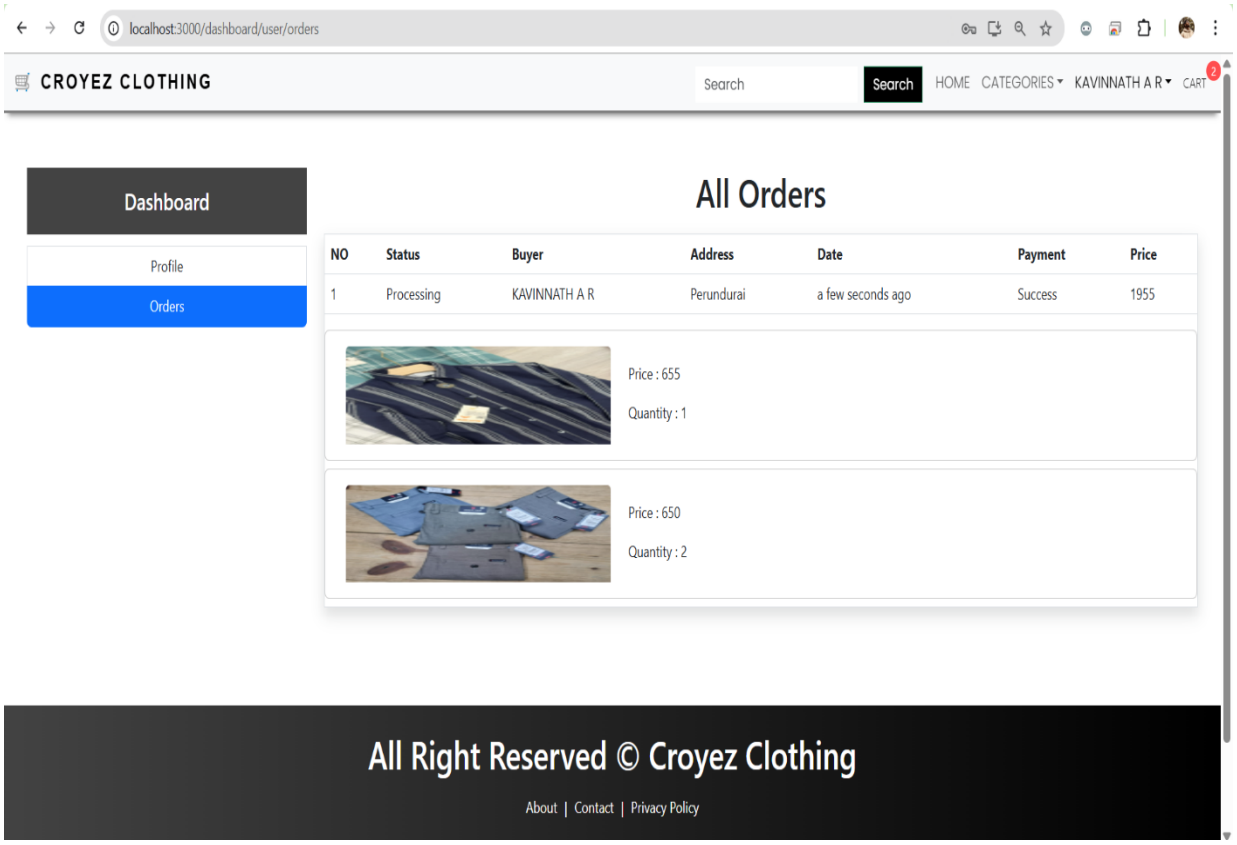


Figure A2.9 ORDERS

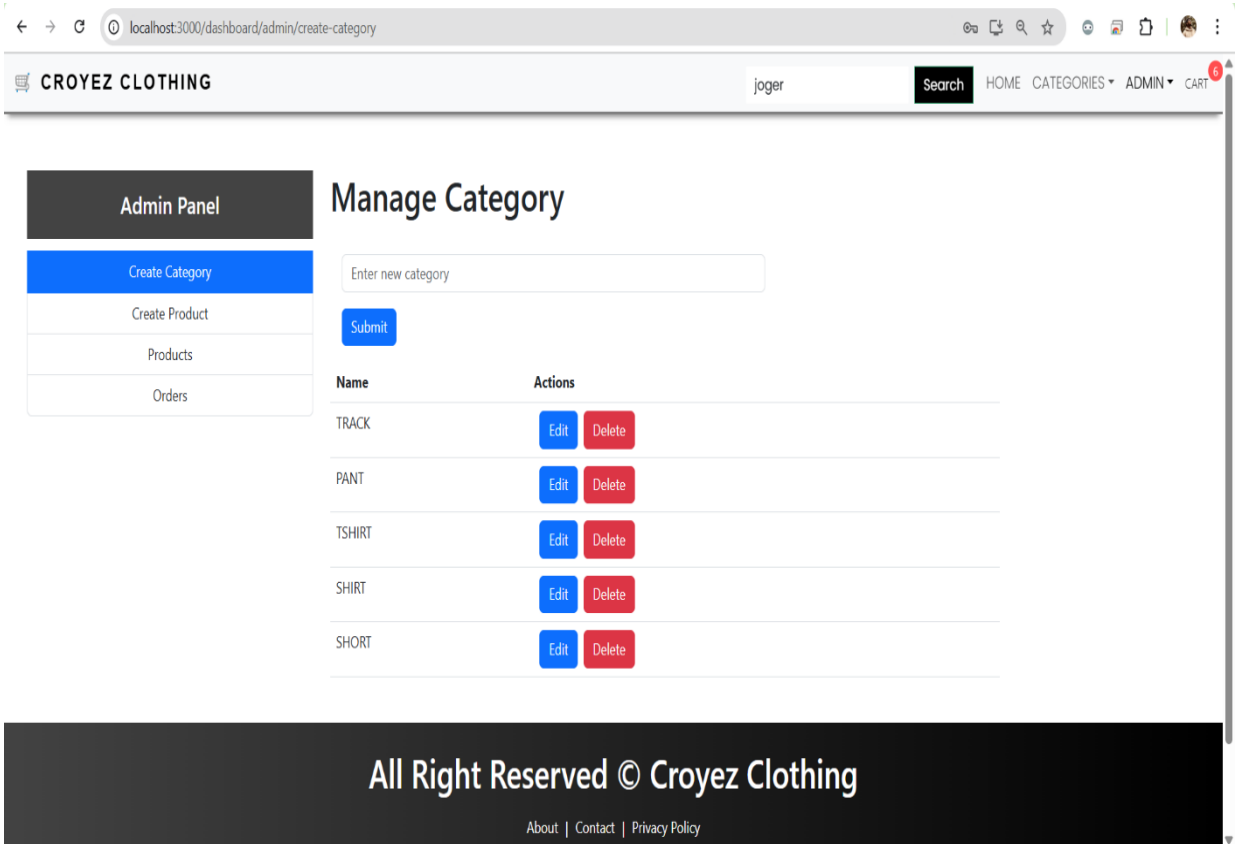


Figure A2.10 MANAGE CATEGORY

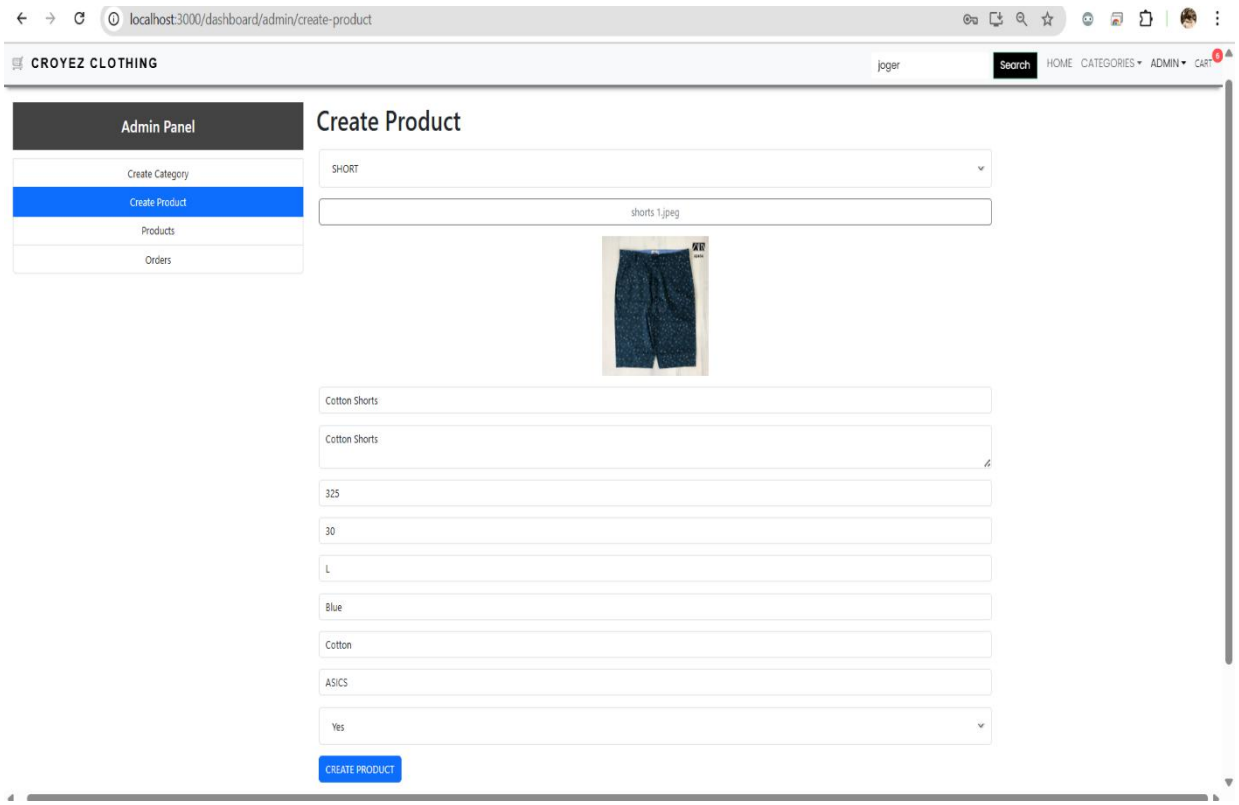


Figure A2.11 CREATE PRODUCT

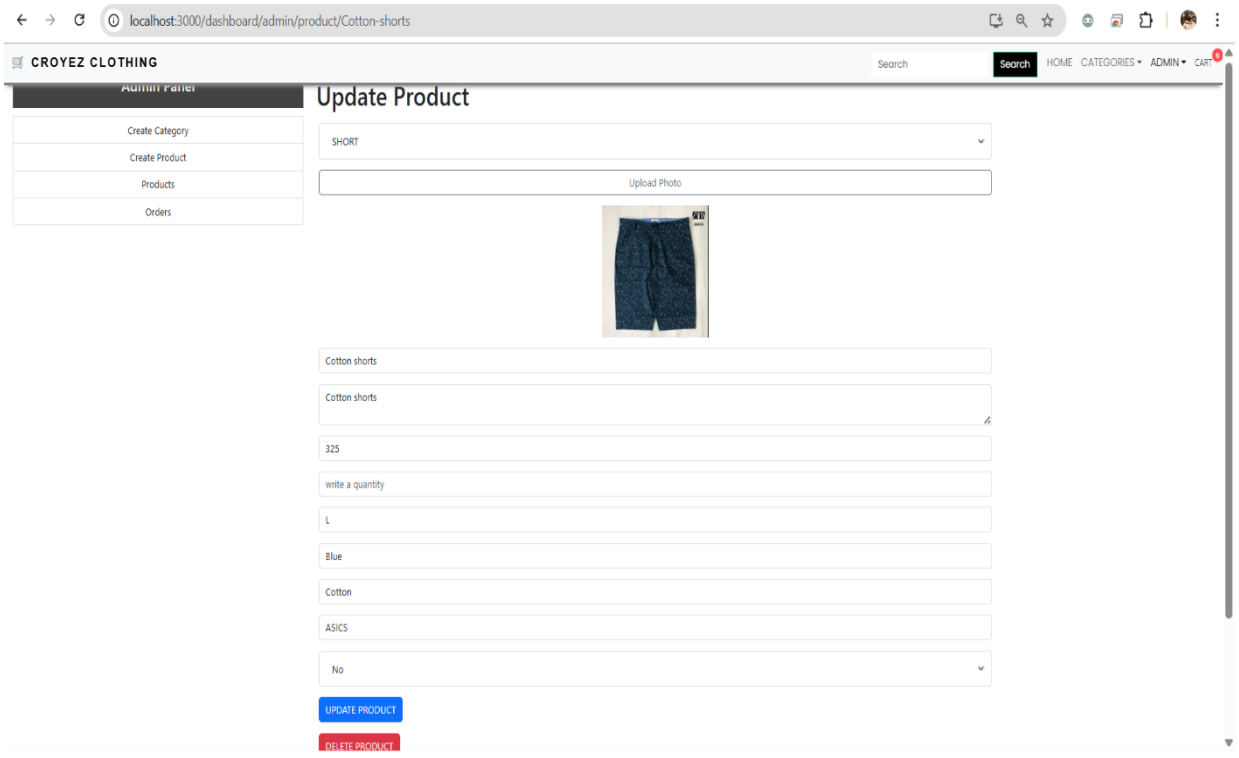


Figure A2.12 UPDATE / DELETE PRODUCT

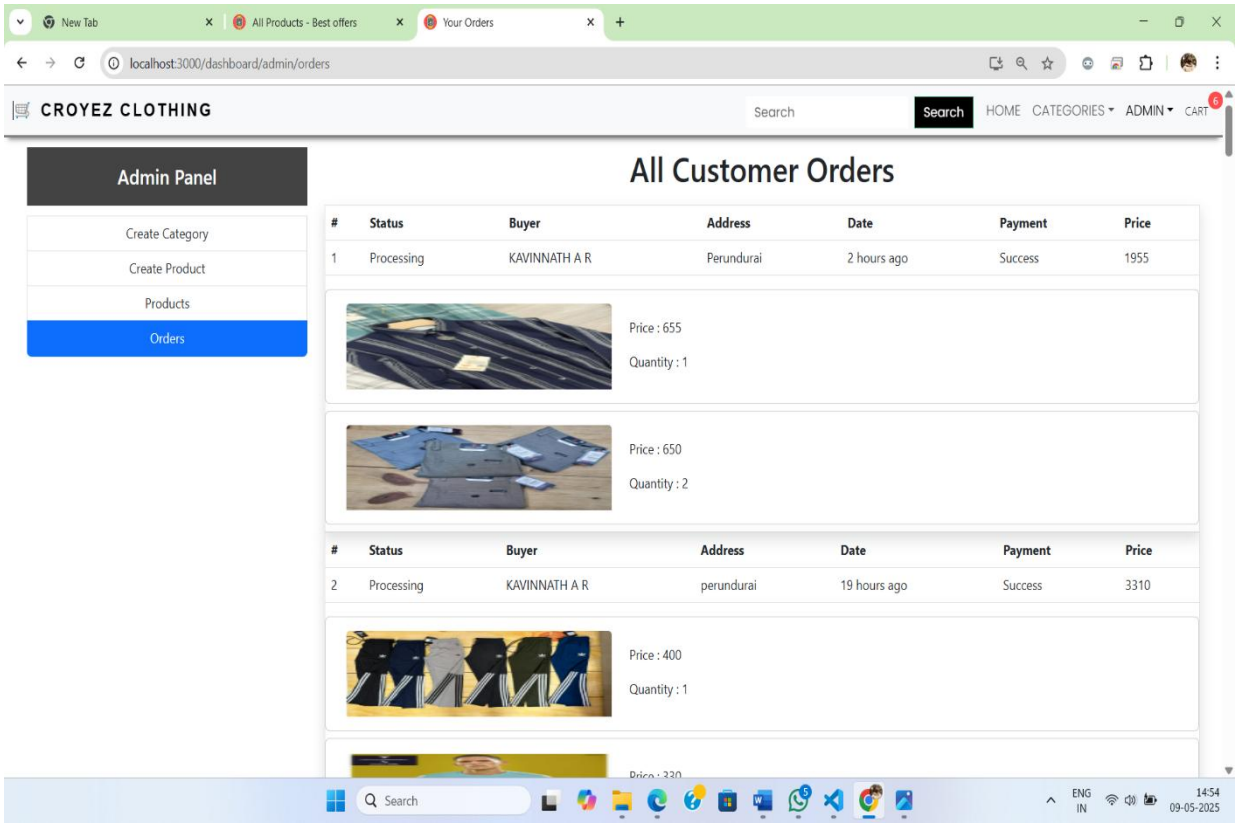


Figure A2.13 ADMIN ORDERS

REFERENCES

- Mehra, Monika, Manish Kumar, Anjali Maurya, and Charu Sharma. "MERN stack Web Development." Annals of the Romanian Society for Cell Biology 25, no. 6 (2021): 11756-11761

- Yogesh Baiskar, Priyas Paulzagade, Krutik Koradia, Pramod Ingole, Dhiraj Shirbhate, "MERN: A Full-Stack Development", International Journal for Research in Applied Science & Engineering Technology (IJRASET) ISSN: 2321- 9653; IC Value: 45.98; SJ Impact Factor: 7.538 Volume 10 Issue I Jan 2022.

- Ranganathan, T.S. Library Management Systems: An Overview. New Delhi: Digital Publishing, 2019.

- Hasanov, Hasan. Developing a Digital Library System. Boston: TechWorks Publishing, 2020.

- Johnson, Claire, and Emily Martin. "Digital Library Systems in Modern Libraries: An Evaluation." Library and Information Science Journal 75, no. 2 (2022): 90-104.

- Anderson, Mark. "Digital Library Management: Trends and Future Directions." Journal of Library Innovation 12, no. 1 (2023): 45-52.

- Williams, Robert, and Sarah Thompson. "User-Centered Design in Library Management Systems: Enhancing Accessibility and Usability." International Journal of Digital Libraries 29, no. 3 (2022): 145-160.